



**Universidad Don Bosco**  
**FACULTAD DE INGENIERÍA**  
**ESCUELA DE COMPUTACIÓN**  
**Desarrollo de aplicaciones con Software Propietario**  
**Guía 7: POO: Implementación de Herencia**



## I. Objetivos

Que el estudiante sea capaz de:

1. Identificar cuándo aplicar herencia dentro de un sistema orientado a objetos.
2. Diseñar jerarquías de clases utilizando clases base y clases derivadas.
3. Implementar correctamente los principios de herencia y polimorfismo en C#.
4. Aplicar clases abstractas y métodos virtuales en el desarrollo de aplicaciones WinForms.

## II. Introducción Teórica

### Herencia y Polimorfismo en Programación Orientada a Objetos (POO)

La programación orientada a objetos (POO) permite que los objetos colaboren entre sí y reutilicen funcionalidades mediante dos características fundamentales: **herencia** y **polimorfismo**. Estas facilitan la eficiencia en el desarrollo y la reducción de redundancias en el código.

#### Herencia

La **herencia** es el mecanismo que permite crear nuevas clases a partir de clases existentes, conocidas como **clases base** o **superclases**. Las clases derivadas heredan atributos, propiedades, métodos y eventos de la clase base, pudiendo además **extender** o **modificar** (override) su comportamiento.

- **Ventajas:** Reutilización de código, reducción de errores y mantenimiento simplificado.
- **Uso recomendado:** Cuando existe una relación de tipo **es-un (is-a)** entre clases. No es adecuado usarla para relaciones **tiene-un (has-a)**.
  - Ejemplo: Automovil → AutomovilDeportivo (is-a).
  - No recomendable: Persona → Automovil (has-a).

#### Clases Base y Derivadas

- **Clase Base:** Define miembros comunes que pueden ser heredados.
- **Clase Derivada:** Hereda y puede modificar o extender los miembros de la clase base.

#### Nota:

- Todas las clases en C# heredan implícitamente de la clase Object.
- C# no permite herencia múltiple; cada clase derivada solo puede tener una clase base.

#### Sintaxis en C#:

```
public class ClaseDerivada : ClaseBase
{
    // miembros adicionales o modificados
}
```

### Modificadores de Acceso

Controlan la visibilidad de los miembros heredados:

| Modificador        | Alcance                                    |
|--------------------|--------------------------------------------|
| public             | Acceso sin restricciones                   |
| protected          | Acceso en la clase y derivadas             |
| internal           | Acceso dentro del ensamblado               |
| protected internal | Acceso en ensamblado o derivadas           |
| private            | Acceso solo dentro de la clase contenedora |

### Clases Especiales

- **Clase sellada (sealed):** No puede ser base de otra clase.

```
public sealed class ClaseSellada { }
```

- **Clase abstracta (abstract):** No puede instanciarse directamente; debe heredarse.

```
public abstract class ClaseAbstracta { }
```

### Polimorfismo y Modificación de Comportamiento

El **polimorfismo** permite que métodos heredados se comporten de manera diferente según la clase que los implemente:

- **Virtual:** Método heredado que puede modificarse en la clase derivada.
- **Abstract:** Método sin implementación en la clase base, que obliga a la clase derivada a implementarlo.

Esto hace que la reutilización de código sea **flexible, adaptable y eficiente**, permitiendo que un mismo método pueda tener múltiples comportamientos según el contexto de la clase que lo utiliza.

## III. Requerimientos

| Nº | Cantidad | Descripción                                                    |
|----|----------|----------------------------------------------------------------|
| 1  | 1        | PC con Microsoft Visual Studio 2026 .NET o superior, instalado |
| 2  | 1        | Guía 7                                                         |
| 3  | 1        | Memoria USB                                                    |

## IV. Procedimiento

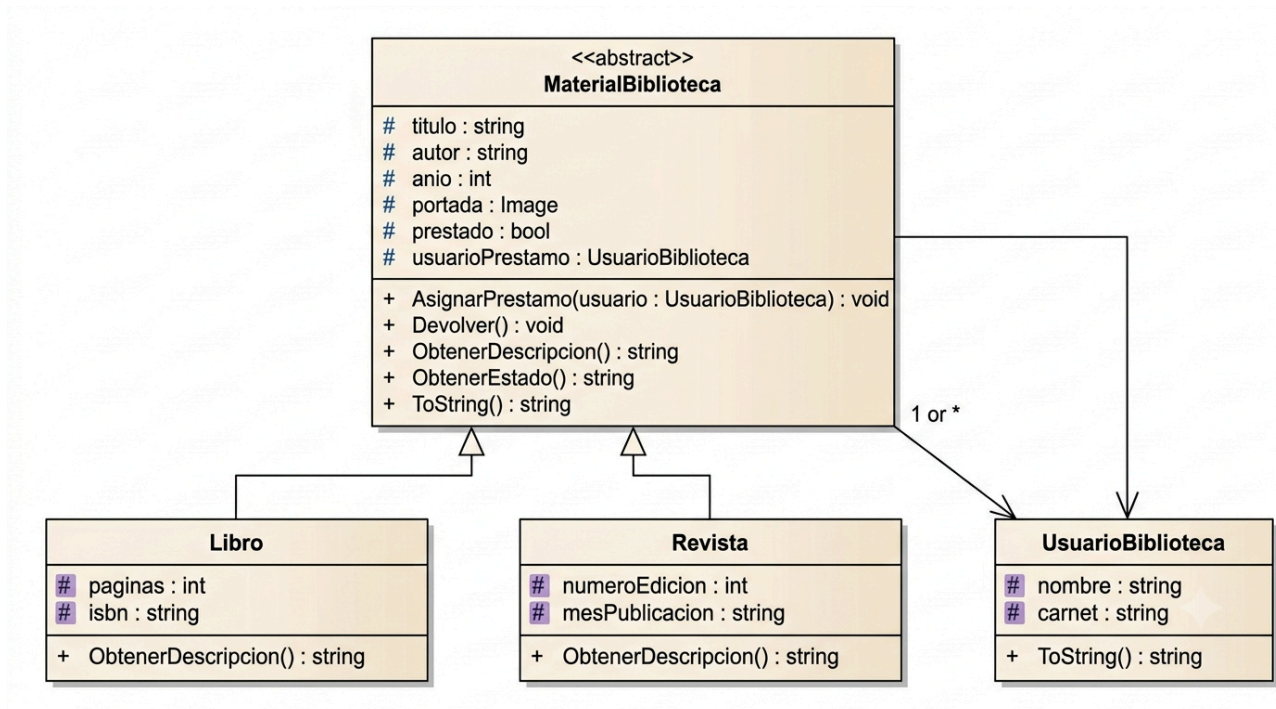
### Problema a Resolver

En una biblioteca académica se requiere un sistema que permita gestionar de forma eficiente el registro de usuarios y el préstamo de materiales bibliográficos. Los materiales disponibles incluyen libros y revistas, los cuales comparten características comunes como título, autor, año de publicación, portada y estado de préstamo. Sin embargo, cada tipo de material posee atributos específicos: los libros tienen número de páginas e ISBN, mientras que las revistas incluyen edición y mes de publicación.

Los usuarios deben poder registrarse con su nombre y número de carnet, y realizar acciones como solicitar préstamos y devolver materiales. El sistema debe permitir aplicar principios de programación orientada a

objetos, como **herencia** para reutilizar atributos comunes entre materiales, y **polimorfismo** para manejar préstamos de distintos tipos de materiales de forma uniforme.

Se ofrece una propuesta de solución. A continuación, desarrollara la implementación de la solución al siguiente problema:



### Ejercicio 1. Crear Formulario de Sistema de Biblioteca

1. Abre Visual Studio.
2. Crea un nuevo proyecto de tipo Aplicación de Consola (.NET Framework) y nómbralo: HerenciaBiblioteca.

#### Definir la clase base: MaterialBiblioteca

1. Crear una nueva clase llamada MaterialBiblioteca. Esta clase será abstracta.  
*NOTA: MaterialBiblioteca se declara abstracta porque representa un concepto general y no debe existir un objeto "MaterialBiblioteca" directamente. Solo deben existir objetos específicos como Libro o Revista.*
2. Esta clase contendrá propiedades comunes a todos los materiales:
  - o Titulo, Autor, Anio, Portada, Prestado, UsuarioPrestamo.
3. Métodos:
  - o AsignarPrestamo(UsuarioBiblioteca usuario)
  - o Devolver()
  - o ObtenerDescripcion() (virtual)
  - o ObtenerEstado()
4. Asegúrate de tener esta línea al inicio de tu archivo: **using System.Drawing;**

### MaterialBiblioteca.cs

```
// Clase base
public abstract class MaterialBiblioteca
{
    public string Titulo { get; set; }
    public string Autor { get; set; }
    public int Anio { get; set; }
    public Image Portada { get; set; }

    public bool Prestado { get; private set; }
    public UsuarioBiblioteca UsuarioPrestamo { get; private set; }

    public MaterialBiblioteca(string titulo, string autor, int anio, Image portada)
    {
        Titulo = titulo;
        Autor = autor;
        Anio = anio;
        Portada = portada;
        Prestado = false;
    }

    public void AsignarPrestamo(UsuarioBiblioteca usuario)
    {
        if (usuario == null)
            throw new ArgumentNullException(nameof(usuario));

        if (Prestado)
            throw new InvalidOperationException( "El material ya se encuentra prestado.");

        Prestado = true;
        UsuarioPrestamo = usuario;
    }

    public void Devolver()
    {
        Prestado = false;
        UsuarioPrestamo = null;
    }

    public abstract string ObtenerDescripcion();

    public string ObtenerEstado()
    {
        if (Prestado && UsuarioPrestamo != null)
            return $"Prestado a {UsuarioPrestamo.Nombre}";
    }
}
```

```
        return "Disponible";
    }

    public override string ToString()
    {
        return Titulo;
    }
}
```

### Crear las clases derivadas: Libro y Revista

1. Crear dos clases nueva clase llamadas Libro y Revista.
2. Ambas heredan de MaterialBiblioteca.
3. Agregan sus propios atributos:
  - o Libro: Paginas, ISBN
  - o Revista: NumeroEdicion, MesPublicacion
4. Sobrescriben el método ObtenerDescripcion().
5. Asegúrate de tener esta línea al inicio de tu archivo: **using System.Drawing;**

#### Libro.cs

```
// Clase hija: Libro
public class Libro : MaterialBiblioteca
{
    public int Paginas { get; set; }
    public string ISBN { get; set; }

    public Libro(string titulo, string autor, int anio, Image portada, int paginas, string isbn)
        : base(titulo, autor, anio, portada)
    {
        Paginas = paginas;
        ISBN = isbn;
    }

    public override string ObtenerDescripcion()
    {
        return $"Libro: {Titulo} - {Autor} ({Anio}), {Paginas} páginas, ISBN: {ISBN}";
    }
}
```

#### Revista.cs

```
// Clase hija: Revista
public class Revista : MaterialBiblioteca
{
    public int NumeroEdicion { get; set; }
    public string MesPublicacion { get; set; }

    public Revista(string titulo, string autor, int anio, Image portada, int edicion, string mes)
```

```
        : base(titulo, autor, anio, portada)
    {
        NumeroEdicion = edicion;
        MesPublicacion = mes;
    }
    public override string ObtenerDescripcion()
    {
        return $"Revista: {Titulo} - {Autor} ({Anio}), Edición {NumeroEdicion}, Mes: {MesPublicacion}";
    }
}
```

#### **UsuarioBiblioteca.cs**

```
public class UsuarioBiblioteca
{
    public string Nombre { get; set; }
    public string Carnet { get; set; }

    public UsuarioBiblioteca(string nombre, string carnet)
    {
        Nombre = nombre;
        Carnet = carnet;
    }
    public override string ToString()
    {
        return $"{Nombre} ({Carnet})";
    }
}
```

## **Ejercicio 2. Implementación lógica del formulario**

1. Ahora diseñe un form con los siguientes controles mostrados en la imagen.
2. Proceda a modificar las propiedades de los controles anteriores, de acuerdo a la tabla de asignaciones a continuación:

**tabPage1**

Form1

Usuarios Prestamos

### Registro de Usuario

Nombre Completo

Camet

Agregar Usuario

tabPage2

Form1

Usuarios Prestamos

### Control de Prestamos

Lista de materiales disponibles

Lista de usuarios

Prestar

Devolver

| Tipo de Control | Nombre del Control | Propiedades a Modificar |
|-----------------|--------------------|-------------------------|
|-----------------|--------------------|-------------------------|

|              |                   |                                                                                       |
|--------------|-------------------|---------------------------------------------------------------------------------------|
| Tabcontrol   | tabBiblioteca     | Tabpages (Colección)<br>Tabpage1.text="Usuarios "<br>Tabpage2.text="Prestamos"        |
| Tabpage1     |                   |                                                                                       |
| Label        | lblUsuario        | Text ="Registro de Usuario"<br>Font.Estilo de fuente = "Negrita"<br>Font.Tamaño="16"  |
| Label        | lblNombreUsuario  | Text = "Nombre Completo"                                                              |
| Label        | lblCarnet         | Text = "Carnet"                                                                       |
| TextBox      | txtNombreUsuario  |                                                                                       |
| TextBox      | txtCarnetUsuario  |                                                                                       |
| Button       | btnAgregarUsuario | Text="Agregar Usuario"                                                                |
| DataGridView | dgvUsuarios       |                                                                                       |
| Tabpage2     |                   |                                                                                       |
| Label        | lblPrestamo       | Text ="Control de Prestamos"<br>Font.Estilo de fuente = "Negrita"<br>Font.Tamaño="16" |
| Label        | lblMaterial       | Text="Lista de materiales disponibles"                                                |
| Label        | lblUsuario        | Text = "Lista de usuarios"                                                            |
| ComboBox     | cmbMaterial       |                                                                                       |
| ComboBox     | cmbUsuario        |                                                                                       |
| Button       | btnPrestar        | Text = "Prestar"                                                                      |
| Button       | btnDevolver       | Text = "Devolver"                                                                     |
| DataGridView | dgvPrestamos      |                                                                                       |

1. Instanciar de forma global los objetos usuario y materiales, con la siguiente línea de código:  
 List<UsuarioBiblioteca> usuarios = new List<UsuarioBiblioteca>();  
 List<MaterialBiblioteca> materiales = new List<MaterialBiblioteca>();
2. Ahora, digite los siguientes bloques de código dentro de c/procedimiento de evento indicado



| Control                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Evento       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| <b>Form1</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |              |
| InitializeComponent();<br>InicializarColumnasUsuarios();<br>InicializarColumnasPrestamos();<br>CargarMaterialesIniciales(); // Simulación de inventario<br>ActualizarMateriales(); // Actualiza ComboBox y DataGridView<br>ActualizarUsuarios(); // Si tienes usuarios precargados<br>ActualizarPrestamos(); // Muestra estado inicial                                                                                                                                                                                                                                                      |              |
| <b>btnPrestar</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | <b>Click</b> |
| <pre> if (cmbMaterial.SelectedItem == null) {     MessageBox.Show("Seleccione un material.");     return; } if (cmbUsuario.SelectedItem == null) {     MessageBox.Show("Seleccione un usuario.");     return; } MaterialBiblioteca material = (MaterialBiblioteca)cmbMaterial.SelectedItem; UsuarioBiblioteca usuario =     (UsuarioBiblioteca)cmbUsuario.SelectedItem; if (material.Prestado) {     MessageBox.Show("El material ya está prestado.");     return; } material.AsignarPrestamo(usuario); ActualizarPrestamos(); MessageBox.Show("Préstamo realizado correctamente."); </pre> |              |
| <b>btnAgregarUsuario</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <b>Click</b> |
| <pre> if (string.IsNullOrEmpty(txtNombreUsuario.Text)) {     MessageBox.Show("Ingrese el nombre del usuario.");     return; } if (string.IsNullOrEmpty(txtCarnetUsuario.Text)) { </pre>                                                                                                                                                                                                                                                                                                                                                                                                     |              |

```

        MessageBox.Show("Ingrese el carnet del usuario.");
        return;
    }
    if (usuarios.Any(u => u.Carnet == txtCarnetUsuario.Text))
    {
        MessageBox.Show("Ya existe un usuario con ese carnet.");
        return;
    }
    var usuario = new UsuarioBiblioteca(
        txtNombreUsuario.Text,
        txtCarnetUsuario.Text);
    usuarios.Add(usuario);
    ActualizarUsuarios();

```

**btnDevolver | Click**

```

if (cmbMaterial.SelectedItem == null)
{
    MessageBox.Show("Seleccione un material.");
    return;
}
MaterialBiblioteca material = (MaterialBiblioteca)cmbMaterial.SelectedItem;
if (!material.Prestado)
{
    MessageBox.Show("El material no tiene préstamo activo.");
    return;
}
material.Devolver();
ActualizarPrestamos();
MessageBox.Show("Material devuelto correctamente.");

```

**private void CargarMaterialesIniciales() | Funcion propia**

```

// Libros
materiales.Add(new Libro("Cien años de soledad", "Gabriel García Márquez", 1967, null, 417, "123-456-789"));
materiales.Add(new Libro("Don Quijote de la Mancha", "Miguel de Cervantes", 1605, null, 863, "987-654-321"));
materiales.Add(new Libro("1984", "George Orwell", 1949, null, 328, "111-222-333"));
materiales.Add(new Libro("El Señor de los Anillos", "J.R.R. Tolkien", 1954, null, 1178, "444-555-666"));
materiales.Add(new Libro("La Odisea", "Homero", -800, null, 541, "777-888-999"));
// Revistas
materiales.Add(new Revista("National Geographic", "Varios", 2023, null, 254, "Marzo"));
materiales.Add(new Revista("Science", "AAAS", 2022, null, 120, "Junio"));
materiales.Add(new Revista("Nature", "Springer", 2021, null, 95, "Agosto"));
materiales.Add(new Revista("Time", "Varios", 2020, null, 80, "Octubre"));
materiales.Add(new Revista("Forbes", "Varios", 2023, null, 150, "Enero"));

```

**private void ActualizarUsuarios() | Funcion propia**

|                                                                                                                                                                                                   |                       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| <pre>dgvUsuarios.Rows.Clear();     cmbUsuario.Items.Clear();     foreach (var u in usuarios)     {         dgvUsuarios.Rows.Add(u.Nombre, u.Carnet);         cmbUsuario.Items.Add(u);     }</pre> |                       |
| <b>Private void ActualizarMateriales()</b>                                                                                                                                                        | <b>Funcion propia</b> |
| <pre>cmbMaterial.Items.Clear();     foreach (var m in materiales)     {         cmbMaterial.Items.Add(m);     }</pre>                                                                             |                       |
| <b>private void ActualizarPrestamos()</b>                                                                                                                                                         | <b>Funcion propia</b> |
| <pre>cmbMaterial.Items.Clear();     foreach (var m in materiales)     {         cmbMaterial.Items.Add(m);     }</pre>                                                                             |                       |
| <b>private void InicializarColumnasUsuarios()</b>                                                                                                                                                 | <b>Funcion propia</b> |
| <pre>dgvUsuarios.Columns.Clear(); dgvUsuarios.Columns.Add("Nombre", "Nombre"); dgvUsuarios.Columns.Add("Carnet", "Carnet");</pre>                                                                 |                       |
| <b>private void InicializarColumnasPrestamos()</b>                                                                                                                                                | <b>Funcion propia</b> |
| <pre>dgvPrestamos.Columns.Clear(); dgvPrestamos.Columns.Add("Titulo", "Título"); dgvPrestamos.Columns.Add("Descripcion", "Descripción"); dgvPrestamos.Columns.Add("Estado", "Estado");</pre>      |                       |

## V. Ejercicios complementarios

El instructor evaluará el desempeño de su trabajo dentro la clase. Estos ejercicios se evaluarán en la misma hora de clase.

1. Implementa un método que devuelva solo los materiales que **no están prestados**.
2. Agrega una función que recorra los materiales y cuente cuántos están prestados a un usuario específico.
3. Crea un método que devuelva todos los materiales prestados por un usuario.
4. Crea una nueva clase llamada Tesis que herede de MaterialBiblioteca y tenga atributos como:
  - a. Universidad
  - b. GradoAcademico

Sobrescribe ObtenerDescripcion() para incluir esta información.

5. Permitir al usuario seleccionar una imagen desde su computadora para asociarla a un libro o a un usuario de biblioteca. Para ello, se debe utilizar el control **PictureBox** para mostrar la imagen

seleccionada, y el componente **OpenFileDialog** para explorar y elegir el archivo de imagen desde el sistema de archivos.

### VI. Análisis de resultados

#### Para la próxima semana

1. Realice una aplicación informática que administre los datos de los diferentes residentes que habitan en un zoológico.
2. Para facilitar la clasificación de los datos, se hará uso de un *esquema de clases*, que se ajustará a la siguiente descripción:
  - a. Crear una clase abstracta denominada **Animales**, la cual tendrá los siguientes datos comunes a todas las especies de animales:
    - i. Nombre común
    - ii. Nombre científico
    - iii. Foto
    - iv. Familia (por ejemplo: carnívoro, ungulados, primates, pájaros, gallinácea, insectos, arácnidos, crustáceos, etc.)
    - v. Hábitat (por ejemplo: selva, desierto, mar, río, lagos, etc....)
    - vi. (Fecha de registro/última actualización)
  - b. De la clase anterior, se deberán crear clases derivadas que permitan registrar a la siguiente información específica de cada residente del zoológico:
    - i. Mamífero: se clasifican por su desarrollo embrionario (monotremas, marsupiales y placentarios) y cantidad de mamas.
    - ii. Reptil: su orden (lagartos, cocodrilos, ofidios, queloneos) y por el tipo de locomoción (patas, garras, rastreros).
    - iii. Peces: se clasifican por su sistema esquelético (óseos, cartilagosos, Actinopterygios) y su desarrollo embrionario (ovíparos, vivíparos)
    - iv. Artrópodos: que debe clasificarlos por tipo terrestre o marino, por su cantidad de patas y cantidad de antenas.
3. Para el registro de los datos, se hará uso del esquema de clases diseñado en numeral anterior, creando una serie de Listas, una por cada tipo de clase derivada.
4. Diseñe una interfaz (pantalla) que facilite a los administradores del zoológico el registro de datos de cada animal que llegue ahí y las actualizaciones posteriores de sus datos.

#### Importante:

- Diseñar la aplicación de manera tal que el usuario deba escribir lo menos posible, dándole prioridad al uso del ratón.
- Permita a los usuarios ver un informe de los animales ya registrados en el sistema, mostrando en un único datagridview a los datos registrados por cada animal.

### V. Bibliografía

1. Microsoft Learn - Documentación oficial de .NET, <https://learn.microsoft.com/es-es/dotnet/> Incluye tutoriales de C#, WinForms, y controles UI.
2. Troelsen, A. (2023). Pro C# 10 with .NET 6. Apress. Libro de referencia para sintaxis avanzada, patrones de diseño y buenas prácticas.
5. Richter, J. (2023). CLR via C#. Microsoft Press. Fundamentos del runtime de .NET y gestión de memoria.
6. Stack Overflow - Comunidad .NET, <https://stackoverflow.com/questions/tagged/c%23>. Resolución de errores comunes y mejores prácticas validadas por la comunidad.
7. DevExpress - WinForms UI Best Practices, <https://www.devexpress.com/winforms/> Guías de diseño de interfaces y uso de controles avanzados.

### Recursos adicionales y Lecturas adicionales

- **Extensión de Visual Studio:**  
[Microsoft XAML Controls Gallery](#) para inspirar diseños modernos.
- **Video-tutoriales:**  
Playlist de YouTube sobre WinForms (ej: "Canal de Programming with Mosh").
- **Libros:**
  - o Troelsen, A. (2022). Pro C# 10 with .NET 6. Apress.
  - o Richter, J. (2020). CLR via C#. Microsoft Press.
- Medrano, K., Hernandez, E., Tejada, R., Gonzalez, B., & Fuentes, N. (2023, July 21). Design of a Prototype System for Real-Time Water Quality Monitoring in Tilapia Farms Using IoT Technology. Presented at the 21st LACCEI International Multi-Conference for Engineering, Education and Technology (LACCEI 2023). <https://doi.org/10.18687/laccei2023.1.1.316>. Disponible en: <http://hdl.handle.net/11715/2757>.
- Medrano, K., Tejada, R., Gonzalez, B., & Juarez, S. (2023, December 10). Aplicación Móvil Interactiva de Realidad Aumentada para la Interacción de Ambientes Culturales. Presented at the 1ra Exposición de Pósteres Científicos y Conferencias Tecnológicas, Universidad Don Bosco, El Salvador. Disponible en: <http://hdl.handle.net/11715/2759>.